Dynamic Text-to-SQL Generator Using Large Language Models Runtime Schema Awareness

Krish Chourasia
IT Department
Acropolis Institute of Technology & Research, Indore
krishchourasia231303@acropolis.in

Lakshya Bhagwat
IT Department
Acropolis Institute of Technology & Research, Indore
Lakshyabhagwat230487@acropolis.in

Prof. Kapil Sahu
IT Department
Acropolis Institute of Technology & Research, Indore
@kapilsahuacropolis.in

Mansij Namdev
IT Department
Acropolis Institute of Technology & Research, Indore
mansijnamdev231187@acropolis.in

Manthan Jadhav
IT Department
Acropolis Institute of Technology & Research, Indore
manthanjadhav230745@acropolis.in

Prof. Ankita Agrawal
IT Department
Acropolis Institute of Technology & Research, Indore
ankitaagrawal@acropolis.in

Abstract — Text-to-SQL systems enable users to interact with relational databases using natural language instead of SQL. This paper presents a Dynamic Text-to-SQL Generator that combines runtime schema extraction, Large Language Models (LLMs), and automated query execution. Unlike conventional approaches that depend on hardcoded database schemas, the proposed system retrieves schema information directly from a live SQLite database before each query. The schema and user prompt are supplied to the DeepSeek-V3 language model through Hugging Face inference APIs, enabling accurate SQL generation. The generated query is executed and returned as structured JSON data. The architecture improves adaptability, transparency, and maintainability while reducing dependence on database experts.

Keywords—Text-to-SQL, Large Language Models, DeepSeek-V3, Natural Language Processing, SQLite, Runtime Schema Awareness, Database Querying.

I. INTRODUCTION

Relational databases store critical information across business, healthcare, education, and e-commerce systems. Accessing this information generally requires SQL expertise, creating a barrier for non-technical users. Text-to-SQL systems aim to bridge this gap by translating natural language questions into executable SQL queries. Recent advances in LLMs have significantly improved Text-to-SQL performance. However, many systems depend on static schemas and require manual updates whenever database structures change. This paper proposes a runtime-aware architecture that dynamically extracts database schemas and uses LLM reasoning for SQL generation.

A. Problem Context

Many organizations rely heavily on relational databases, which serve as the backbone of most data-driven systems. However, querying them effectively demands absolute proficiency in SQL, a language that remains precise, unforgiving, and unfamiliar to individuals outside of software and data engineering. This discrepancy generates a substantial access gap. Business analysts, operations teams, researchers, and general end users who urgently require data are heavily dependent on developers to write queries for them. This persistent dependency severely slows operational decisions and restricts who can directly work with data.

B. Motivation and Contributions

The primary motivation of this project is built around a single foundational insight: if a capable language model is provided with an accurate, up-to-date description of a database's structure alongside a user's question, it can generate correct SQL reliably. Rather than hardcoding the database structure directly into the application layer, this architecture reads the schema dynamically from the live database at the exact time of each request. The system's primary contributions include:

- It establishes a runtime schema-aware pipeline that queries internal metadata directly from live databases.
- It eliminates hardcoded SQL templates, supporting arbitrary natural language inputs.
- It provides full transparency by surfacing generated SQL queries directly to the user alongside the results.
- It decouples the AI inference layer from the data layer, allowing independent model updates or schema alterations.

II. LITERATURE REVIEW

Recent research demonstrates the effectiveness of LLMs in Text-to-SQL tasks. DAIL-SQL achieved state-of-the-art performance through advanced prompt engineering and example selection strategies. Existing approaches focus on schema representation, prompt optimization, and supervised fine-tuning. Despite high accuracy, many systems rely on

predefined schemas and benchmark datasets. Runtime schema-aware architectures remain relatively underexplored. This work builds upon LLM-based query generation while emphasizing adaptability to evolving databases.

III. PROBLEM STATEMENT

Organizations frequently maintain databases that evolve over time. Traditional Text-to-SQL systems often fail when schemas change because table definitions are embedded within application logic. Existing approaches to this problem suffer from two distinct issues: they either require a fixed, pre-defined schema to be hardcoded into the system, making them brittle when the database changes, or they use generic models with no awareness of the specific database they are operating on, leading to inaccurate or hallucinated queries. Users without SQL knowledge remain dependent on technical staff for data retrieval. The objective is to design a system capable of automatically understanding database structure and generating valid SQL queries without requiring manual schema updates in the code.

IV. PROPOSED METHODOLOGY

The proposed methodology is structured into a dynamic, execution-ready sequence that translates natural language inquiries into real-time database outputs. Every user request passes through three explicit stages.

A. Overall System Framework

The execution pipeline is initiated immediately upon receiving a natural language prompt from the frontend UI, such as: "Show me the names of customers who placed orders for products costing more than $50". The internal breakdown of these operational stages is detailed below in Table I.

TABLE I. TEXT-TO-SQL THREE-STAGE PROCESSING PIPELINE

Stage	Name	Description
Stage 1	Schema Extraction	The system reads the live database's internal metadata (such as sqlite_master) to retrieve all table definitions, columns, types, and relationships as they currently exist.
Stage 2	LLM Inference	The extracted schema and the user's natural language question are formatted into a prompt and submitted to DeepSeek-V3. The model reasons over the schema and generates a raw SQL query.
Stage 3	Query Execution & Response	The generated SQL query is executed directly against the live SQLite database, and results are returned as structured JSON.

No SQL queries are permanently stored or pre-templated; the code is generated completely fresh for every request.

V. SYSTEM ARCHITECTURE

The architecture consists of three core layers orchestrated to handle decoupled execution streams. The interaction flow spans from the front-facing client app to the back-end processing service, and finally out to the remote intelligence model:

- Frontend (React): A single-page web application featuring a dual-screen design. It includes a landing page detailing the application capabilities and an active query interface where users input text, observe the generated SQL, and view structured results tables.
- Backend (Spring Boot): A Java microservice that coordinates pipeline actions. It manages inbound endpoints, reads database metadata, builds contextual LLM prompts, processes API communications, executes queries, and structures outbound data payloads.
- External AI Service (Hugging Face): Hosts the language model infrastructure, accepting requests via the Chat Completions API format to perform natural language reasoning and SQL generation using DeepSeek-V3.

VI. TECHNOLOGY STACK

The full-stack application relies entirely on open-source technologies and modern architectural frameworks. The concrete deployment technologies are categorized by layer in Table II.

TABLE II. SYSTEM TECHNOLOGY STACK

Layer	Technology	Role
Frontend	React 18, Vite, Tailwind CSS	User interface construction and query submission.
Backend	Java 21, Spring Boot 3.5.7	API hosting and end-to-end pipeline orchestration.

AI / LLM	DeepSeek-V3 via Hugging Face	Natural language reasoning and SQL generation.
Database	SQLite	Data storage, metadata tracking, and query execution.
HTTP Client	Spring WebFlux (WebClient)	Non-blocking asynchronous calls to the LLM API.
Build Tool	Maven	Dependency management and application packaging.

VII. DATABASE DESIGN

The prototype system operates on a pre-loaded transactional SQLite database (database.db) representing a classic e-commerce domain shared globally across all system users. The system contains five core tables, structured to evaluate advanced multi-table joins, filtering operations, and aggregate queries.

TABLE III. E-COMMERCE DATA MODEL SCHEMA

Table Name	Description
customers	Tracks registered users, storing names, countries, and specific signup dates.
products	Holds the product catalogue, indexing item names, categories, and prices.
orders	Tracks purchase transactions placed individually by customers.
order_items	Acts as a cross-reference table mapping line items within each order.
payments	Stores transactional financial data records tied back to specific orders.

VIII. ADVANTAGES AND DISTINCT FEATURES

The proposed architecture demonstrates clear functional milestones over legacy rule-based or template-bound code pipelines:

- Runtime Schema Awareness: Schema properties are never hardcoded into application properties; instead, they are polled from sqlite_master during runtime on every unique request, rendering the application immune to post-deployment database structural changes.
- No Query Templates: The model generates raw SQL dynamically from scratch instead of matching text against rigid, pre-configured query patterns.
- Full Transparency: Exposing the generated SQL directly alongside data tables empowers developers to audit model logic and helps users verify performance transparency.
- Decoupled Design: Complete separation between the LLM inference provider and the storage engine allows developers to swap language models or update databases independently.

IX. RESULT AND DISCUSSION

Experimental evaluation demonstrates successful conversion of natural language questions into executable SQL statements. Runtime schema extraction enables adaptation to schema modifications without code changes. The architecture improves maintainability and transparency because the generated SQL is visible to users. The system performs effectively for joins, aggregations, filtering operations, and multi-table queries.

X. LIMITATIONS

While the architecture provides extreme flexibility, the system depends on external LLM inference services, introducing latency. Detailed operational constraints include:

- LLM Latency: Inference processing introduces an operational delay of 2–6 seconds per query, making it less ideal for high-frequency or real-time production loops.
- Query Safety Scope: The system is built around read-only SELECT statements; there are currently no safety validation layers to actively block destructive commands such as INSERT, UPDATE, or DELETE.
- Model Bounded Quality: Distorted, complex, or ambiguous user queries can cause the LLM to hallucinate incorrect column names or formulate invalid SQL queries.
- No Multi-Tenancy: The application targets a single shared database structure without multi-user isolation or individual database provisioning.
- Lack of Access Controls: The backend exposes an open API endpoint without integrated user authentication or request rate limiting.

XI. CONCLUSION

The Dynamic Text-to-SQL Generator demonstrates the practical application of LLMs for database accessibility. By combining runtime schema extraction with DeepSeek-V3, the

system offers an adaptive and scalable solution for natural language database interaction. A final descriptive summary of the research architecture is outlined in Table IV.

TABLE IV. PROJECT AT A GLANCE

Attribute	Value
Project Type	Full-stack AI-powered web application.
Primary Focus Area	Natural Language Processing, Database Systems.
Core Functionality	Text-to-SQL translation and direct query execution.
Backend Language	Java 21 (Spring Boot 3.5.7 framework).
Frontend Framework	React 18 (Vite build tool, Tailwind CSS styling).
AI Processing Model	DeepSeek-V3 via Hugging Face Inference API.
Database Component	SQLite (Single shared transactional file).
API Interface Style	REST (Single operational GET endpoint).
System License	MIT License.

XII. FUTURE WORK

Future work targets resolving production constraints to scale the proof-of-concept system into an enterprise microservice. Key enhancements include adding multi-database routing support, interactive conversational refinement queries, automated query safety verification, integrated security controls, rigorous metric benchmarking across multiple vendor LLMs, and result explanation generation using automated natural language text summaries.

XIII. REFERENCES

[1] D. Gao et al., "Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation," PVLDB, 2024.

[2] OpenAI, GPT-4 Technical Report.

[3] Hugging Face Inference API Documentation.

[4] Spring Boot Documentation.

[5] SQLite Documentation.

[6] Amanda Askell, Yuntao Bai, Anna Chen, Dawn Drain, Deep Ganguli, Tom Henighan, Andy Jones, Nicholas Joseph, Benjamin Mann, Nova DasSarma, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Jackson Kernion, Kamal Ndousse, Catherine Olsson, Dario Amodei, Tom B. Brown, Jack Clark, Sam McCandlish, Chris Olah, and Jared Kaplan. 2021. A General Language Assistant as a Laboratory for Alignment. CoRR abs/2112.00861 (2021).

[7] LILY Group at Yale University. 2018. Spider 1.0, Yale Semantic Parsing and Text-to-SQL Challenge. https://yale-lily.github.io/spider.

[8] Christopher Baik, Zhongjun Jin, Michael J. Cafarella, and H. V. Jagadish. 2020. Duoquest: A Dual-Specification System for Expressive SQL Queries. In Proceedings of the 2020 International Conference on Management of Data. 2319–2329.

[9] Ursin Brunner and Kurt Stockinger. 2021. ValueNet: A Natural Language-toSQL System that Learns from Database Information. In 37th IEEE International Conference on Data Engineering. 2177–2182.

[10] Ruichu Cai, Boyan Xu, Zhenjie Zhang, Xiaoyan Yang, Zijian Li, and Zhihao Liang. 2018. An Encoder-Decoder Framework Translating Natural Language to Database Queries. In Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence. 3977–3983.

[11] Shuaichen Chang and Eric Fosler-Lussier. 2023. How to Prompt LLMs for Textto-SQL: A Study in Zero-shot, Single-domain, and Cross-domain Settings. CoRR abs/2305.11853 (2023).

[12] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. 2023. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality. https://lmsys.org/blog/2023-03-30-vicuna/

[13] Naihao Deng, Yulong Chen, and Yue Zhang. 2022. Recent Advances in Text-toSQL: A Survey of What We Have and What We Expect. In Proceedings of the 29th International Conference on Computational Linguistics. 2166–2187.

[14] Xiang Deng, Ahmed Hassan Awadallah, Christopher Meek, Oleksandr Polozov, Huan Sun, and Matthew Richardson. 2021. Structure-Grounded Pretraining for Text-to-SQL. In Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. 1337–1350.

[15] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. 4171–4186.